РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

дисциплина: Архитектура компьютера

Студент: Моисеев Дмитрий Вячеславович

Группа: НБИбд-02-25

МОСКВА

2026 г.

Цель работы

- Изучить идеологию и применение средств контроля версий.
- Освоить умения по работе с git.

Базовая настройка git

Первичная настройка параметров git

- Зададим имя и email владельца репозитория:
- `git config --global user.name "Name Surname"`
- `git config --global user.email "work@mail"`
- Настроим utf-8 в выводе сообщений git:
- `git config --global core.quotepath false`
- Настройте верификацию и подписание коммитов git.
- Зададим имя начальной ветки (будем называть её `master`):
- `git config --global init.defaultBranch master`

Учёт переносов строк

- В разных операционных системах приняты разные символы для перевода строк:
  - Windows: `\r\n` (`CR` и `LF`);
  - Unix: `\n` (`LF`);
  - Mac: `\r` (`CR`).
- Посмотреть значения переносов строк в репозитории можно командой:
- `git ls-files --eol`

1. Параметр `autocrlf`
   - Настройка `core.autocrlf` предназначена для того, чтобы в главном репозитории все переводы строк текстовых файлах были одинаковы.
   - Настройка `core.autocrlf` с параметрами `true` и `input` делает все переводы строк текстовых файлов в главном репозитории одинаковыми.
     - `core.autocrlf true`: конвертация `CRLF->LF` при коммите и обратно `LF->CRLF` при выгрузке кода из репозитория на файловую систему (обычно используется в Windows).
     - `core.autocrlf input`: конвертация `CRLF->LF` только при коммитах (используются в MacOS/Linux).

Варианты конвертации

Таблица 1.: Варианты конвертации для разных значений параметра core.autocrlf

core.autocrlf	false	input	true
git commit	LF -> LF	LF -> LF	LF -> CRLF

Таблица 1.: Варианты конвертации для разных значений параметра core.autocrlf

core.autocrlf	false	input	true
	CR -> CR	CR -> CR	CR -> CR
	CRLF -> CRLF	CRLF -> LF	CRLF -> CRLF
git checkout	LF -> LF	LF -> LF	LF -> CRLF
	CR -> CR	CR -> CR	CR -> CR
	CRLF -> CRLF	CRLF -> CRLF	CRLF -> CRLF

1.
   - Установка параметра:
     - Для Windows
     - `git config --global core.autocrlf true`
     - Для Linux
     - `git config --global core.autocrlf input`
2. Параметр `safecrlf`
   - Настройка `core.safecrlf` предназначена для проверки, является ли окончаний строк обратимым для текущей настройки `core.autocrlf`.
     - `core.safecrlf true`: запрещается необратимое преобразование `lf<->crlf`. Полезно, когда существуют бинарные файлы, похожие на текстовые файлы.
     - `core.safecrlf warn`: печать предупреждения, но коммиты с необратимым переходом принимаются.
   - Установка параметра:
   - `git config --global core.safecrlf warn`

Создание ключа ssh

Общая информация

1. Алгоритмы шифрования ssh
   1. Аутентификация

      В SSH поддерживается четыре алгоритма аутентификации по открытым ключам:

      - DSA:
        - размер ключей DSA не может превышать 1024, его следует отключить;
      - RSA:
        - следует создавать ключ большого размера: 4096 бит;
      - ECDSA:
        - ECDSA завязан на технологиях NIST, его следует отключить;
      - Ed25519:
        - используется пока не везде.
   2. Симметричные шифры

        - Из 15 поддерживаемых в SSH алгоритмов симметричного шифрования, безопасными можно считать:
            - `chacha20-poly1305`;
            - `aes*-ctr`;
            - `aes*-gcm`.
        - Шифры `3des-cbc` и `arcfour` потенциально уязвимы в силу использования DES и RC4.
        - Шифр `cast128-cbc` применяет слишком короткий размер блока (64 бит).
    3. Обмен ключами
        - Применяемые в SSH методы обмена ключей DH (Diffie-Hellman) и ECDH (Elliptic Curve Diffie-Hellman) можно считать безопасными.
        - Из 8 поддерживаемых в SSH протоколов обмена ключами вызывают подозрения три, основанные на рекомендациях NIST:
            - `ecdh-sha2-nistp256`;
            - `ecdh-sha2-nistp384`;
            - `ecdh-sha2-nistp521`.
        - Не стоит использовать протоколы, основанные на *SHA1*.
2. Файлы ssh-ключей
    - По умолчанию пользовательские ssh-ключи сохраняются в каталоге `~/.ssh` в домашнем каталоге пользователя.
    - Убедитесь, что у вас ещё нет ключа.
    - Файлы закрытых ключей имеют названия типа `id_<алгоритм>` (например, `id_dsa`, `id_rsa`).
        - По умолчанию закрытые ключи имеют имена:
        - `id_dsa`
        - `id_ecdsa`
        - `id_ed25519`
        - `id_rsa`
    - Открытые ключи имеют дополнительные расширения `.pub`.
        - По умолчанию публичные ключи имеют имена:
        - `id_dsa.pub`
        - `id_ecdsa.pub`
        - `id_ed25519.pub`
        - `id_rsa.pub`
    - При создании ключа команда попросит ввести любую ключевую фразу для более надёжной защиты вашего пароля. Можно пропустить этот этап, нажав `Enter`.
    - Сменить пароль на ключ можно с помощью команды:
    - `ssh-keygen -p`

Создание ключа ssh

- Ключ ssh создаётся командой:
- `ssh-keygen -t <алгоритм>`
- Создайте ключи:
    - по алгоритму *rsa* с ключём размером 4096 бит:
    - `ssh-keygen -t rsa -b 4096`
    - по алгоритму *ed25519*:
    - `ssh-keygen -t ed25519`
- При создании ключа команда попросит ввести любую ключевую фразу для более надёжной защиты вашего пароля. Можно пропустить этот этап, нажав `Enter`.
- Сменить пароль на ключ можно с помощью команды:
- `ssh-keygen -p`

Добавление SSH-ключа в учётную запись GitHub

- Скопируйте созданный SSH-ключ в буфер обмена командой:
- `xclip -i < ~/.ssh/id_ed25519.pub`
- Откройте настройки своего аккаунта на GitHub и перейдем в раздел `SSH and GPC keys`.
- Нажмите кнопку `ew SSH key`.
- Добавьте в поле `Title` название этого ключа, например, `ed25519@hostname`.
- Вставьте из буфера обмена в поле `Key` ключ.
- Нажмите кнопку `Add SSH key`.

Верификация коммитов с помощью *PGP*

- Как настроить PGP-подпись коммитов с помощью `gpg`.

Общая информация

- Коммиты имеют следующие свойства:
  - author (автор) — контрибьютор, выполнивший работу (указывается для справки);
  - committer (коммитер) — пользователь, который закоммитил изменения.
- Эти свойства можно переопределить при совершении коммита.
- Авторство коммита можно подделать.
- В git есть функция подписи коммитов.
- Для подписывания коммитов используется технология PGP (см. [Работа с PGP]).
- Подпись коммита позволяет удостовериться в том, кто является коммитером. Авторство не проверяется.

Создание ключа

- Генерируем ключ
- `gpg --full-generate-key`
- Из предложенных опций выбираем:
  - тип *RSA and RSA*;
  - размер 4096;
  - выберите срок действия; значение по умолчанию — 0 (срок действия не истекает никогда).
- GPG запросит личную информацию, которая сохранится в ключе:
  - Имя (не менее 5 символов).
  - Адрес электронной почты.
    - При вводе email убедитесь, что он соответствует адресу, используемому на GitHub.
  - Комментарий. Можно ввести что угодно или нажать клавишу ввода, чтобы оставить это поле пустым.

Экспорт ключа

- Выводим список ключей и копируем отпечаток приватного ключа:
- `gpg --list-secret-keys --keyid-format LONG`
- Отпечаток ключа — это последовательность байтов, используемая для идентификации более длинного, по сравнению с самим отпечатком ключа.
- Формат строки:
- `sec  Алгоритм/Отпечаток_ключа Дата_создания [Флаги] [Годен_до]`

- `ID_ключа`
- Экспортируем ключ в формате ASCII по его отпечатку:
- `gpg --armor --export <PGP Fingerprint>`

Добавление PGP ключа в GitHub

- Копируем ключ и добавляем его в настройках профиля на GitHub (или GitLab).
- Скопируйте ваш сгенерированный PGP ключ в буфер обмена:
- `gpg --armor --export <PGP Fingerprint> | xclip -sel clip`
- Перейдите в настройки GitHub (https://github.com/settings/keys), нажмите на кнопку *New GPG key* и вставьте полученный ключ в поле ввода.

Подписывание коммитов git

- Подпись коммитов при работе через терминал:
- `git commit -a -S -m 'your commit message'`
- Флаг `-S` означает создание подписанного коммита. При этом может потребоваться ввод кодовой фразы, заданной при генерации GPG-ключа.

Настройка автоматических подписей коммитов git

- Используя введёный email, укажите Git применять его при подписи коммитов:
- `git config --global user.signingkey <PGP Fingerprint>`
- `git config --global commit.gpgsign true`
- `git config --global gpg.program $(which gpg2)`

Проверка коммитов в Git

- GitHub и GitLab будут показывать значок *Verified* рядом с вашими новыми коммитами.

Режим бдительности (vigilant mode)

- На GitHub есть настройка vigilant mode.
- Все неподписанные коммиты будут явно помечены как *Unverified*.
- Включается это в настройках в разделе *SSH and GPG keys*. Установите метку на *Flag unsigned commits as unverified*.

Задание

- Создать базовую конфигурацию для работы с git.
- Создать ключ *SSH*.
- Создать ключ *PGP*.
- Настроить подписи git.
- Зарегистрироваться на *Github*.
- Создать локальный каталог для выполнения заданий по предмету.

Последовательность выполнения работы

Установка программного обеспечения

Установка git

- Установим *git*:
- `dnf install git`

Установка gh

- Fedora:
- `dnf install gh`

Базовая настройка git

- Зададим имя и email владельца репозитория:
- `git config --global user.name "Name Surname"`
- `git config --global user.email "work@mail"`
- Настроим utf-8 в выводе сообщений git:
- `git config --global core.quotepath false`
- Настройте верификацию и подписание коммитов git (см. [Верификация коммитов git с помощью GPG]).
- Зададим имя начальной ветки (будем называть её `master`):
- `git config --global init.defaultBranch master`
- Параметр `autocrlf`:
- `git config --global core.autocrlf input`
- Параметр `safecrlf`:
- `git config --global core.safecrlf warn`

Создайте ключи *ssh*

- по алгоритму *rsa* с ключём размером 4096 бит:
- `ssh-keygen -t rsa -b 4096`
- по алгоритму *ed25519*:
- `ssh-keygen -t ed25519`

Создайте ключи *pgp*

- Генерируем ключ
- `gpg --full-generate-key`
- Из предложенных опций выбираем:
  - тип *RSA and RSA*;
  - размер 4096;
  - выберите срок действия; значение по умолчанию — 0 (срок действия не истекает никогда).
- GPG запросит личную информацию, которая сохранится в ключе:
  - Имя (не менее 5 символов).
  - Адрес электронной почты.
    - При вводе email убедитесь, что он соответствует адресу, используемому на GitHub.

    - Комментарий. Можно ввести что угодно или нажать клавишу ввода, чтобы оставить это поле пустым.

Настройка github

- Создайте учётную запись на https://github.com.
- Заполните основные данные на https://github.com.

Добавление PGP ключа в GitHub

- Выводим список ключей и копируем отпечаток приватного ключа:
- `gpg --list-secret-keys --keyid-format LONG`
- Отпечаток ключа — это последовательность байтов, используемая для идентификации более длинного, по сравнению с самим отпечатком ключа.
- Формат строки:
- `sec  Алгоритм/Отпечаток_ключа Дата_создания [Флаги] [Годен_до]`
- `ID_ключа`
- Скопируйте ваш сгенерированный PGP ключ в буфер обмена:
- `gpg --armor --export <PGP Fingerprint> | xclip -sel clip`
- Перейдите в настройки GitHub (https://github.com/settings/keys), нажмите на кнопку *New GPG key* и вставьте полученный ключ в поле ввода.

Настройка автоматических подписей коммитов git

- Используя введёный email, укажите Git применять его при подписи коммитов:
- `git config --global user.signingkey <PGP Fingerprint>`
- `git config --global commit.gpgsign true`
- `git config --global gpg.program $(which gpg2)`

Настройка gh

- Для начала необходимо авторизоваться
- `gh auth login`
- Утилита задаст несколько наводящих вопросов.
- Авторизоваться можно через броузер.

Шаблон для рабочего пространства

- Рабочее пространство для лабораторной работы
- Репозиторий: https://github.com/yamadharma/course-directory-student-template.

Сознание репозитория курса на основе шаблона

- Необходимо создать шаблон рабочего пространства (см. Рабочее пространство для лабораторной работы).
- Например, для 2022–2023 учебного года и предмета «Операционные системы» (код предмета `os-intro`) создание репозитория примет следующий вид:
- `mkdir -p ~/work/study/2022-2023/"Операционные системы"`
- `cd ~/work/study/2022-2023/"Операционные системы"`

- `gh repo create study_2022-2023_os-intro --template=yamadharma/course-directory-student-template --public`
- `git clone --recursive git@github.com:<owner>/study_2022-2023_os-intro.git os-intro`

Настройка каталога курса

- Перейдите в каталог курса:
- `cd ~/work/study/2022-2023/"Операционные системы"/os-intro`
- Удалите лишние файлы:
- `rm package.json`
- Создайте необходимые каталоги:
- `echo os-intro > COURSE`
- `make`
- Отправьте файлы на сервер:
- `git add .`
- `git commit -am 'feat(main): make course structure'`
  `git push`

Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

- Система контроля версий (VCS) — это ПО для управления изменениями в коде или документах.
  Задачи:
- Сохранение истории изменений (кто, когда и что изменил).
- Возможность отката к любой предыдущей версии.
- Совместная работа нескольких человек над одним проектом без риска затереть чужой код.

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

- Хранилище (Репозиторий): база данных, где VCS хранит всю историю изменений и метаданные.
- Рабочая копия: текущее состояние файлов проекта на диске пользователя, с которыми он непосредственно работает.
- Commit (Коммит): фиксация изменений. Снимок состояния файлов в определенный момент времени с уникальным ID.
- История: цепочка коммитов, позволяющая проследить развитие проекта.

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

- Центлизованные: есть один сервер с репозиторием. Пользователи скачивают только рабочую копию. Если сервер недоступен, история недоступна. Пример: Subversion (SVN).
- Децентрализованные: каждый пользователь имеет полную копию всего репозитория (включая всю историю). Работа возможна без интернета. Пример: Git, Mercurial.

4. Опишите действия с VCS при единоличной работе с хранилищем.

- Инициализация репозитория.
- Внесение изменений в файлы.
- Индексация изменений (подготовка к сохранению).

5. Опишите порядок работы с общим хранилищем VCS.

- Клонирование (или обновление) актуальной версии из общего хранилища.
- Внесение локальных правок и создание коммитов.
- Слияние (Pull/Fetch) чужих изменений, если они появились, пока вы работали.
- Отправка (Push) своих коммитов в общее хранилище.

6. Каковы основные задачи, решаемые инструментальным средством `git`?

- Обеспечение целостности данных (через хеширование).
- Высокая скорость работы с локальными операциями.
- Поддержка нелинейной разработки (тысячи параллельных веток).

7. Назовите и дайте краткую характеристику командам `git`.

- git init: создание нового репозитория.
- git clone: копирование удаленного репозитория.

- git add: добавление файлов в индекс (stage).
- git commit: фиксация проиндексированных изменений.
- git status: просмотр состояния рабочей копии.
- git push / git pull: отправка / получение данных из удаленного репозитория.
- git log: просмотр истории.

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

- Локально: git add main.py -> git commit -m "Fix bug" (сохранили правку у себя).
- Удалённо: git push origin master (отправили локальный код на сервер GitHub/GitLab).

9. Что такое и зачем могут быть нужны ветви (branches)?

- Ветви — это указатели на определенный коммит, позволяющие вести разработку параллельно.
  Зачем нужны: для изоляции работы над новой функцией или исправления бага, чтобы основной код (master/main) всегда оставался рабочим. После завершения задачи ветку «вливают» (merge) в основную.

10. Как и зачем можно игнорировать некоторые файлы при commit?

- Для этого используется файл .gitignore.
  Зачем нужно: чтобы в историю не попадал «мусор» — временные файлы, кеши, скомпилированные бинарники или конфиги с секретными паролями, которые не должны быть публичными.
- Нужно ли подготовить для вас пример содержимого файла .gitignore для конкретного языка программирования (например, Python или C++)?

Вывод

В ходе работы я изучил принципы работы **систем контроля версий** и освоил базовые команды **Git**